



Návrh a realizace multifunkčních digitálních hodin s měřením teploty a upozorněním na události

Maturitní práce

Autor	Matěj Duda
Obor	Elektrotechnika
Vedoucí práce	Ing. Jiří Jelínek, Ph.D.
Školní rok	2025/2026
Počet stran	37
Počet slov	6040

Přihláška k maturitní práci

Jméno a příjmení studenta

Duda, Matěj

Název práce

Návrh a realizace multifunkčních digitálních hodin s měřením teploty a upozorněním na události

Třída

E4A

Školní rok

MP2025/26

Přidělené role

Vedoucí práce

Oponent

Podpis

Jelínek, Jiří

Podpis

Mader, Zbyněk

Obecná ustanovení	Vypracování a odevzdání práce proběhne v souladu s platnými normami (vyhláška 177/2009 Sb.) a aktuálním dokumentem "Pokyny k vypracování prací" vydaným školou.
	Práce bude hodnocena z hlediska jejího praktického využití, zvládnutí dokumentace po věcné i formální stránce a obhajoby celé práce. Student byl seznámen s kritérii hodnocení maturitní práce.
	Práce bude odevzdána ve dvou stejnopisech vázaných pevnou nebo kroužkovou vazbou.
	Veškeré náklady na MP včetně vyhotovení obou tištěných kopií si student hradí sám.
Licenční ujednání	Ve smyslu § 60 (Školní dílo) autorského zákona č. 121/2000 Sb. poskytují SPŠSE a VOŠ Liberec výhradní a neomezená práva k využití této mé maturitní práce.
	Bez svolení školy se zdržím jakéhokoliv komerčního využití mé práce.
	Pro výukové účely a prezentaci školy se vzdávám nároku na odměnu za užití díla.

Finanční rozvaha - odhad celkových nákladů

V Kč	Náklady celkem	Hrazené školou
Výrobní	0	0
Na služby	0	0

Jedná se o MP, jejíž vypracování si škola vyžádala? Ano – **Ne**

Podpis studenta (vyjadřuje souhlas s uvedenými údaji a ujednáními)

V Liberci 10.10.2025

Podpis

Konzultant

Práci podporuji

Podpis

Předmětová komise

Práci doporučuji

Podpis

Třídní učitel

Práci doporučuji

Podpis

Garant oboru

Práci doporučuji

Podpis

Ředitel školy

Práci doporučuji

Podpis

Zadání maturitní práce

Název

Návrh a realizace multifunkčních digitálních hodin s měřením teploty a upozorněním na události

Předmět

MIT, TEZ, ELT,
RAZ, AUT, IKT

Téma

Cílem je navrhnout a realizovat multifunkční zařízení pro zobrazení času, které kromě základních hodinových funkcí poskytuje také budík, zobrazení data, měření vnitřní a vnější teploty, audiovizuální upozornění na definované události a přepínatelné režimy provozu (například noční nebo úsporný režim). Výsledkem bude samostatně fungující zařízení využitelné jako chytré domácí hodiny s další rozšířenou funkcí.

Použité prostředky

RGB LED displej, Arduino (ESP případně Raspberry Pi), rádiové připojení venkovního teploměru, teplotní senzory, FM rádio, Bluetooth, Wi-Fi, Ethernet, 3D tisk, bezdrátové nabíjení.

Cíle práce

1	Navrhnout schéma zapojení pro multifunkční digitální hodiny s mikroprocesorovým řízením.
2	Navrhnout uživatelské rozhraní ovládání multifunkčních digitálních hodin.
3	Navrhnout přehledné uživatelské rozhraní umožňující ovládání hodin, nastavení budíku, režimů a zobrazení doplňkových funkcí.
4	Připojit senzory pro měření vnitřní a vnější teploty a zobrazit hodnoty na displeji.
5	Implementovat audiovizuální upozornění na definované události (např. notifikace).
6	Zavést přepínatelné provozní režimy, například noční režim nebo úsporu energie.
7	Přidat funkci rádia (alespoň FM) nebo Bluetooth zvuku.

Osnova práce

1	Rešerše možností návrhu chytrých digitálních hodin a přehled dostupných funkcí.
2	Výběr vhodného mikrokontroléru a ostatních komponent (displej, senzory, modul BT/FM).
3	Návrh elektronického zapojení a ovládací logiky.
4	Vývoj softwaru pro zobrazování času, data a obsluhu budíku.
5	Integrace teplotních čidel a zpracování měřených hodnot.
6	Implementace audiovizuálních upozornění a připojení modulu notifikací.
7	Realizace přepínání provozních režimů (např. noční a úsporný).
8	Návrh a výroba mechanické konstrukce (např. 3D tisk).
9	Oživení, testování, ladění funkce zařízení.
10	Vyhodnocení výsledků a návrh možného rozšíření.

Anotace

Tato maturitní práce se zabývá návrhem a realizací multifunkčních digitálních hodin, které fungují jako koncové zařízení internetu věcí (IoT). Práce popisuje kompletní vývoj hardwarové i softwarové části systému. Hardwarová část je postavena na mikrokontroléru Arduino Mega 2560 s připojením k lokální síti pomocí Ethernet Shieldu. Pro zobrazování informací jsou využity LED maticové displeje s řadičem MAX7219 a pro akustickou signalizaci zesilovač LM386 s reproduktorem. Zařízení je dále vybaveno bezdrátovou nabíječkou a je napájeno přes step-down měnič LM2596.

Nedílnou součástí projektu je vlastní serverová aplikace napsaná v jazyce Python (framework Flask) zajišťuje získávání meteorologických dat z externího API, správu budíků a příjem notifikací z mobilního zařízení (Apple Shortcuts). Práce detailně rozebírá teoretické principy použitých technologií, mechanický návrh pomocí 3D tisku a vyhodnocení energetické náročnosti zařízení.

Summary

This graduation thesis deals with the design and realization of a multifunctional digital clock operating as an Internet of Things (IoT) end device. The thesis describes the complete development of both the hardware and software parts of the system. The hardware is based on the Arduino Mega 2560 microcontroller connected to a local network using an Ethernet Shield. Information is displayed using LED matrix displays with the MAX7219 controller, and acoustic signaling is handled by the LM386 amplifier. The device is also equipped with a wireless charger and powered via an LM2596 step-down converter.

An integral part of the project is the custom server application written in Python (Flask framework) ensures the retrieval of meteorological data from an external API, alarm management, and the reception of notifications from a mobile device (Apple Shortcuts). The thesis details the theoretical principles of the used technologies, mechanical design using 3D printing, and the evaluation of the device's energy consumption.

Poděkování

Tímto bych chtěl poděkovat panu učiteli Jiřímu Jelínkovi za odborné vedení práce, cenné rady, metodickou podporu a čas věnovaný konzultacím v průběhu jejího zpracování.

Čestné prohlášení

Prohlašuji, že jsem předkládanou maturitní práci vypracoval sám a uvedl jsem veškerou použitou literaturu a bibliografické citace.

V Liberci dne 16.03.2026

.....

Matěj Duda

Obsah

Poděkování.....	5
Úvod.....	1
1 Koncept internetu věcí (IoT) a architektura systému.....	2
1.1 Architektura mikrokontrolérů.....	3
1.1.1 Platforma Arduino a mikrokontrolér ATmega2560.....	4
1.2 Zobrazovací technologie a LED matice.....	5
1.2.1 Princip multiplexování.....	5
1.2.2 Integrovaný obvod MAX7219.....	6
1.3 Komunikační sběrnice a sítě.....	7
1.3.1 Sériová periferní sběrnice (SPI).....	7
1.3.2 Ethernet a rodina protokolů TCP/IP.....	8
1.3.3 Sítová vrstva a protokol DHCP.....	9
1.3.4 Protokol HTTP a aplikační rozhraní (API).....	9
1.4 Softwarová architektura a serverové technologie.....	10
1.4.1 Jazyk Python a framework Flask.....	10
1.4.2 Integrace externích dat a notifikací (Open-Meteo, iOS Shortcuts).....	10
1.5 Akustická signalizace a zpracování zvuku.....	11
1.5.1 Pulzně šířková modulace (PWM) a obvod LM386.....	11
1.6 Napájení a energetická bilance.....	12
1.6.1 Spínané měniče napětí a obvod LM2596.....	12
1.6.2 Technologie bezdrátového nabíjení (Qi).....	13
1.7 Mechanická konstrukce a 3D tisk.....	13
1.7.1 Návrh v softwaru SolidWorks.....	13
1.7.2 Technologie FDM a materiál PLA.....	14
2 Praktická část – návrh a realizace.....	15
2.1 Blokové schéma a celková architektura systému.....	15
2.2 Hardwarová realizace a zapojení komponent.....	16
2.3 Vývoj serverové aplikace (Backend).....	17
2.3.1 Získávání meteorologických dat.....	17
2.3.2 Správa notifikací a aplikační rozhraní.....	18
2.4 Vývoj firmwaru pro mikrokontrolér.....	18
2.4.1 Inicializace sítě a HTTP komunikace.....	19

2.4.2	Parsování datových struktur bez použití objektu String	20
2.4.3	Správa paměti a vykreslování na LED matice	20
2.5	Mechanický návrh a výroba šasi	22
3	Testování a měření.....	23
3.1	Oživení systému a kalibrace.....	23
3.2	Měření energetické spotřeby.....	24
3.3	Testování softwarové stability.....	25
	Závěr.....	26
	Seznam zkratk a odborných výrazů.....	27
	Seznam obrázků.....	28
	Použité zdroje	29

Úvod

Rozvoj informačních technologií, miniaturizace elektroniky a zvyšující se dostupnost širokopásmového připojení v posledním desetiletí vedly k masivnímu nástupu zařízení začleněných do takzvaného internetu věcí (IoT). Běžné domácí spotřebiče a předměty denní potřeby jsou dnes vybavovány mikrokontroléry a síťovým připojením, což jim umožňuje komunikovat s okolím, stahovat data a poskytovat uživatelům interaktivní a automatizované funkce.

Tradiční digitální hodiny sloužily v domácnostech primárně k jedinému účelu – zobrazení aktuálního času. S příchodem chytrých telefonů a chytrých hodinek se však jejich role začala vytrácet. Cílem této maturitní práce je vytvořit moderní pojetí digitálních hodin, které kromě své primární funkce nabídnou výraznou přidanou hodnotu. Zařízení je navrženo tak, aby vizualizovalo aktuální meteorologická data, umožnilo synchronizaci budíků přes vzdálené webové rozhraní a přímo zobrazovalo notifikace a události z chytrého telefonu (např. ze sociálních sítí či komunikačních aplikací).

Samotné hardwarové hodiny tvoří pouze zobrazovací a výkonný terminál postavený na platformě Arduino Mega 2560. Výpočetně náročné úlohy, jako je stahování šifrovaných dat o počasí a zpracování webhooků z mobilního telefonu, probíhají na dedikovaném serveru ve veřejné síti napsaném v jazyce Python.

1 Koncept internetu věcí (IoT) a architektura systému

Internet věcí (Internet of Things) je technologický koncept označující propojení fyzických zařízení s internetem nebo lokální sítí [2][5]. Tato zařízení jsou vybavena elektronikou, softwarem a síťovým rozhraním, které jim umožňují sbírat a vyměňovat data bez nutnosti přímého lidského zásahu. IoT architektura se typicky dělí do tří základních vrstev:

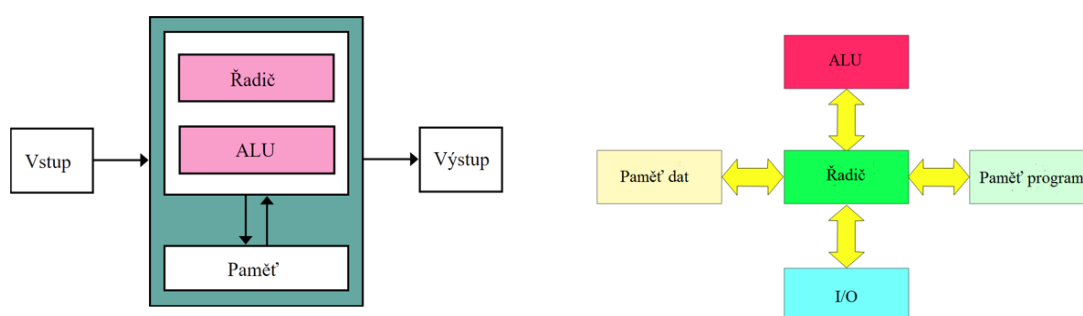
- **Vrstva vnímání (Perception Layer):** Senzory a aktuátory, které sbírají data z fyzického světa nebo do něj zasahují.
- **Síťová vrstva (Network Layer):** Zodpovídá za bezpečný a spolehlivý přenos dat z vrstvy vnímání do systémů zpracování informací (např. přes WiFi nebo Ethernet).
- **Aplikační vrstva (Application Layer):** Poskytuje koncové služby uživateli, analyzuje data a zajišťuje grafické rozhraní.

V kontextu této práce tvoří navržené hodiny koncový uzel IoT sítě (Edge device). Místo přímého oslovování cloudových služeb (např. pro získání počasí), které by pro 8bitový mikrokontrolér představovalo obrovskou zátěž kvůli nutnosti šifrování HTTPS a parsování velkých JSON souborů, je zvolena architektura klient-server využívající vzdálený dedikovaný Oracle server. Tento výkonný cloudový server přebírá aplikační vrstvu, stahuje data z externích API a poskytuje je hodinám v optimalizovaném a snadno parsovatelném formátu.

1.1 Architektura mikrokontrolérů

Mikrokontrolér (MCU - Microcontroller Unit) je kompaktní počítač integrovaný do jediného polovodičového čipu. Obsahuje jedno nebo více procesorových jader, paměť pro uchování programu (typicky Flash), operační paměť pro dočasná data (SRAM nebo EEPROM) a řadu programovatelných vstupně-výstupních (I/O) periférií [1][6].

Na rozdíl od mikroprocesorů v osobních počítačích, které vyžadují externí paměti a řadiče, jsou mikrokontroléry navrženy pro vestavěné (embedded) aplikace. Důraz je u nich kladen na deterministické chování, nízkou spotřebu elektrické energie, vysokou spolehlivost a schopnost přímého řízení hardwaru na nízké úrovni (čtení stavu logických úrovní, generování PWM, ...).



Obr.1 – blokové schéma architektury mikrokontrolerů

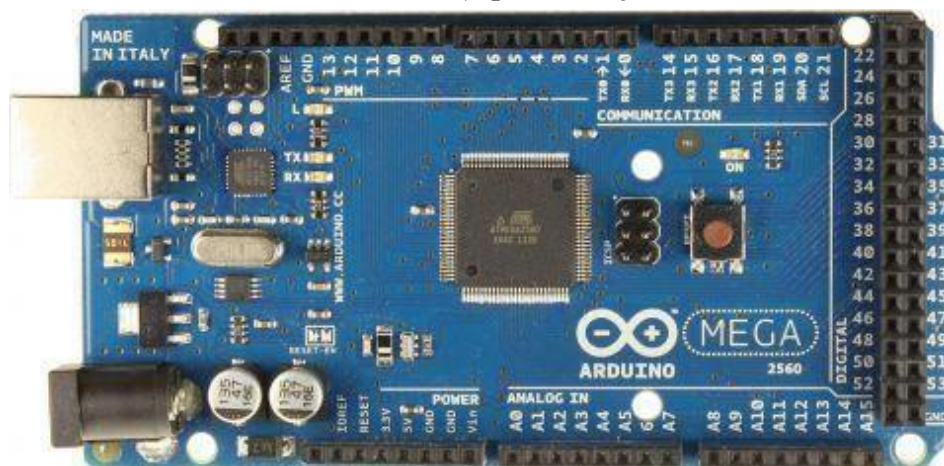
1.1.1 Platforma Arduino a mikrokontrolér ATmega2560

Arduino je celosvětově rozšířená open-source hardwarová a softwarová platforma, která zásadním způsobem zjednodušuje vývoj prototypů elektronických zařízení [3]. Desky Arduino umožňují složité hardwarové nastavení registrů pomocí srozumitelného jazyka založeného na C/C++ (Wiring).

Pro realizaci tohoto projektu byla z široké nabídky vývojových desek zvolena varianta Arduino Mega 2560. Důvodem je náročnost projektu na paměť a množství připojovaných periférií. Jádrem desky je 8bitový mikrokontrolér ATmega2560 od společnosti Atmel (nyní Microchip Technology) s harvardskou architekturou RISC (Reduced Instruction Set Computer), operující na frekvenci 16 MHz.

Klíčové parametry mikrokontroléru ATmega2560, které vedly k jeho výběru oproti standardnímu Arduino Uno (ATmega328P):

- **Paměťová kapacita:** Mikrokontrolér disponuje 256 KB Flash paměti (pro uložení kódu programu a statických polí, např. znakových sad fontů) a 8 KB paměti SRAM. Obsluha TCP/IP stacku pro komunikaci s Ethernet Shieldem, parsování přijatých dat od serveru a udržování datového pole (framebufferu) pro osm modulů 8x8 LED matic vyžaduje značné množství operační paměti. U modelu Uno s pouhými 2 KB SRAM by při takovém objemu dat běžně docházelo k přetečení zásobníku (Stack overflow) a pádům programu.
- **Rozšířený počet I/O pinů:** Deska nabízí celkem 54 digitálních vstupně/výstupních pinů (z nichž 15 lze využít jako PWM výstupy) a 16 analogových vstupů. Nezávislé řízení dvou samostatných řad LED displejů, komunikace se síťovým modulem a generování zvuku vyžaduje dostatek nezávislých pinů bez nutnosti jejich složitějšího multiplexování nebo sdílení.
- **Čtyři hardwarové UARTy:** Obsahuje čtyři nezávislé sériové linky, což zjednodušuje proces ladění programu (jeden UART může posílat data do počítače, zatímco ostatní obsluhují periferie).



Obr.2 – Arduino Mega 2560

1.2 Zobrazovací technologie a LED matice

Zobrazování informací o čase, teplotě, datu a textu notifikací je v projektu realizováno pomocí dvou sestav LED maticových displejů. Každá sestava se skládá ze čtyř fyzických modulů s rozlišením 8x8 světelných bodů (LED diod), čímž vznikají dva bloky o celkovém rozlišení 32x8 pixelů (tzv. 4-in-1 moduly).



Obr.3 - LED Matrix matice 8x8x4 s MAX7219

1.2.1 Princip multiplexování

LED matice 8x8 obsahuje celkem 64 svítivých diod. Kdybychom chtěli každou diodu ovládat individuálně statickým napětím, potřebovali bychom 64 výstupních pinů mikrokontroléru, což je pro praktické použití nereálné. Řešením tohoto problému je technika zvaná multiplexování [4].

V zapojení LED matice jsou anody (nebo katody) diod v jednom řádku propojeny dohromady a opačné póly jsou propojeny ve sloupcích. K ovládní celé matice tak stačí 16 pinů (8 pro řádky a 8 pro sloupce). Rozsvícení konkrétního bodu se provede přivedením napětí na příslušný řádek a uzemněním příslušného sloupce. V jeden okamžik je možné plně ovládat pouze jeden řádek (nebo sloupec) diod. Multiplexování spočívá ve velmi rychlém postupném zapínání a vypínání jednotlivých řádků. Rychlost tohoto přepínání musí být vyšší než zhruba 100 Hz. Díky fyziologickému jevu zvanému setrvačnost lidského oka (Persistence of vision) se jednotlivé záblesky slévají a pozorovatel vnímá statický a celistvý obraz.

1.2.2 Integrovaný obvod MAX7219

Pokud by se o multiplexování, výpočty a načasování signálů staral přímo mikrokontrolér ATmega2560, spotřeboval by pro zobrazení obrazu veškerý svůj strojový čas a nemohl by provádět jiné činnosti, jako je síťová komunikace. Z tohoto důvodu je použit dedikovaný zobrazovací řadič, integrovaný obvod MAX7219 [5].

MAX7219 je kompaktní, sériově řízený driver se společnou katodou, navržený pro řízení mikroprocesorových displejů s až 64 segmenty (nebo jedné matice 8x8).

- **Princip činnosti:** Mikrokontrolér po sériové lince pouze odešle povel a data obsahující informaci o tom, jaké body mají svítit (tzv. framebuffer). MAX7219 si tato data uloží do své interní statické paměti a následně na hardwarové úrovni zcela nezávisle provádí samotné multiplexování (cyklické spínání řádků a sloupců s frekvencí 800 Hz).
- **Řízení jasu:** Obvod nabízí možnost plynulého řízení jasu displeje bez nutnosti přidávat externí předřadné rezistory ke každé LED. Jas lze řídit jednak změnou jednoho hlavního hardwarového rezistoru, a také softwarově v 16 krocích pomocí interní pulzně šířkové modulace (PWM).
- **Kaskádování (Daisy-chaining):** Jednou z největších výhod obvodu MAX7219 je přítomnost pinu Data Out (DOUT). Obvody lze zapojovat do série za sebe. Data z mikrokontroléru vstupují do pinu DIN prvního modulu, projdou posuvným registrem a přes DOUT pokračují do dalšího modulu. Tímto způsobem lze pomocí pouhých 3 řídících vodičů z Arduina obsluhovat neomezené množství matic. Pro dosažení vyšší obnovovací frekvence při plynulém pohybu textu byly však v návrhu matice rozděleny na dvě nezávislé větve (horní pro čas a spodní pro teplotu), z nichž každá má svou vlastní datovou linku.

1.3 Komunikační sběrnice a sítě

1.3.1 Sériová periferní sběrnice (SPI)

Sériové periferní rozhraní (SPI - Serial Peripheral Interface) je synchronní sériová komunikační sběrnice operující v režimu plného duplexu (obousměrný přenos dat současně), navržená společností Motorola. Je primárně využívána pro komunikaci mikrokontroléru s periferními obvody na krátké vzdálenosti.

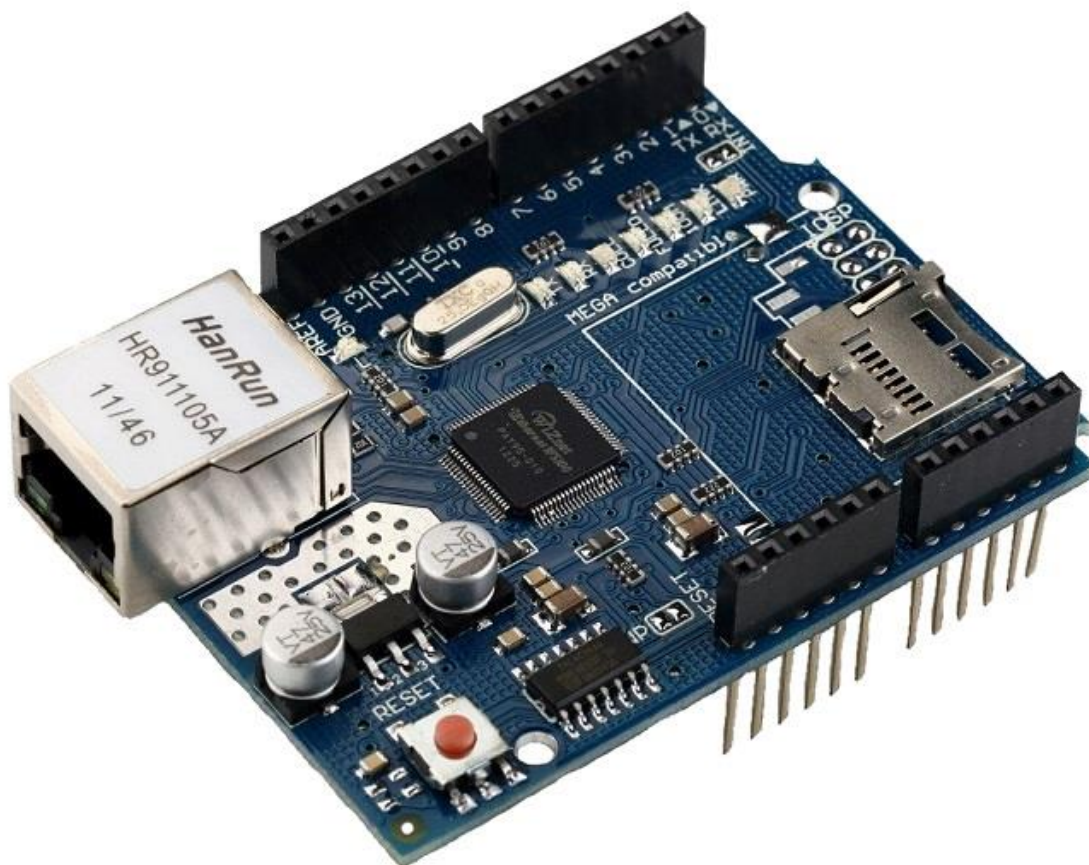
V projektu pomocí SPI komunikuje Arduino s řadiči MAX7219 a s Ethernetovým modulem. Sběrnice se skládá z následujících logických signálů:

- **SCLK (Serial Clock):** Hodinový signál, který generuje Master a udává tempo komunikace.
- **MOSI (Master Out Slave In):** Linka pro přenos dat z mikrokontroléru do periferie.
- **MISO (Master In Slave Out):** Linka pro přenos dat z periferie do mikrokontroléru (obvody MAX7219 tento pin nevyužívají, jelikož data do Arduina nevracejí).
- **SS/CS (Slave Select / Chip Select):** Linka, kterou Master adresuje konkrétní připojené zařízení. Každá periferie musí mít svůj vlastní CS pin. Běžně je udržován ve vysoké úrovni (HIGH), při zahájení komunikace jej Master stáhne na nízkou úroveň (LOW).

1.3.2 Ethernet a rodina protokolů TCP/IP

Pro připojení zařízení do místní sítě LAN (Local Area Network) a zajištění komunikace s lokálním serverem byl zvolen standard drátového Ethernetu IEEE 802.3. Toto připojení je realizováno pomocí rozšiřujícího modulu (tzv. Ethernet Shieldu) kompatibilního s deskou Arduino Mega. Srdcem tohoto modulu je integrovaný obvod Wiznet W5100.

Oproti mnoha levnějším ethernetovým řadičům, které poskytují pouze fyzickou (PHY) a linkovou (MAC) vrstvu, má obvod W5100 naprostou výhodu v hardwarově implementovaném zásobníku síťových protokolů TCP/IP [6]. Samotný čip obstarává logiku protokolů TCP, UDP, IPv4, ICMP, ARP a IGMP. Pro 8bitové Arduino by softwarová implementace těchto složitých síťových vrstev znamenala vyčerpání téměř veškerého výpočetního výkonu a operační paměti. Díky modulu W5100 zapisuje mikrokontrolér přes SPI sběrnici do registrů čipu pouze čistá aplikační data (např. samotný text HTTP požadavku) a čip se postará o vytvoření paketů, zabezpečení spojení a jejich odeslání do sítě.



Obr.4 – Ethernet shield pro Arduino

1.3.3 Sítová vrstva a protokol DHCP

Aby mohlo zařízení na síti TCP/IP komunikovat, musí mít přiřazenu unikátní IP adresu, masku podsítě a bránu. K automatické konfiguraci těchto parametrů slouží protokol DHCP (Dynamic Host Configuration Protocol). Systém po svém startu odešle formou broadcastu do sítě žádost DHCPDISCOVER. Domácí router (sloužící jako DHCP server) mu odpoví a přidělí volnou IP adresu z definovaného rozsahu. Tento přístup výrazně usnadňuje nasazení hodin do jakékoliv sítě bez nutnosti ručně přepisovat IP adresu ve zdrojovém kódu Arduina.

1.3.4 Protokol HTTP a aplikační rozhraní (API)

Komunikace mezi Arduinem a Python serverem probíhá na nejvyšší aplikační vrstvě pomocí protokolu HTTP (Hypertext Transfer Protocol). Mikrokontrolér vystupuje v roli klienta, který periodicky (každých 10 sekund) vytváří TCP spojení na konkrétní IP adresu a port serveru a odesílá metodu GET /api/arduino HTTP/1.1.

Pro přenos stavových dat zpět do Arduina bylo nutné navrhnout co nejjednodušší formát dat. Z důvodu optimalizace zpracování a ochrany paměti není využíván moderní, avšak syntakticky složitý formát JSON, jehož parsování je v C++ značně náročné. Místo toho byla navržena proprietární strukturovaná řetězcová odpověď (tzv. plain-text) s jasně definovanými oddělovači. Odpověď serveru má tvar například:

`<TIME:15:30|DATE:05.10.|TEMP:22|ALARM:0|MSG:Nová zpráva>.`

Program v Arduinu následně čte příchozí bajty tak dlouho, dokud nenarazí na počáteční znak „<“. Znak „|“ slouží jako oddělovač jednotlivých proměnných a znak „>“ komunikaci ukončuje. Extrakce hodnot pak probíhá prostým hledáním klíčových slov (např. "TEMP:") a překopírováním příslušného bloku znaků do proměnné.

1.4 Softwarová architektura a serverové technologie

Jak již bylo zmíněno, k minimalizaci hardwarových nároků koncového zařízení byla většina složité logiky přenesena na dedikovaný lokální server.

1.4.1 Jazyk Python a framework Flask

Serverová aplikace byla vytvořena v programovacím jazyce Python, který vyniká přehledností a masivní podporou knihoven pro zpracování dat a síťovou komunikaci. Pro rychlé sestavení webového serveru a definování API byl využit aplikační mikro-framework Flask [7]. Ten poskytuje tzv. routování (mapování URL adres na konkrétní funkce v kódu) a obsluhu protokolů dle standardu WSGI (Web Server Gateway Interface).

Klíčovým aspektem serverové architektury je využití více vláken (Multithreading). Zatímco hlavní vlákno běží ve smyčce, naslouchá na portu 5000 a poskytuje HTML rozhraní nebo odpovídá Arduino, nezávislé vlákno spuštěné na pozadí periodicky kontroluje přesný čas budíků a dotazuje se na meteorologická data, aniž by došlo k blokování obsluhy hodin.

1.4.2 Integrace externích dat a notifikací (Open-Meteo, iOS

Shortcuts)

Původním záměrem projektu bylo měřit teplotu pomocí fyzických senzorů. Během realizace však došlo k optimalizaci a teplota je získávána pomocí externí služby.

Meteorologická data: Server pomocí knihovny requests odesílá zabezpečený dotaz s přesnými GPS souřadnicemi Liberce na veřejně dostupné API služby Open-Meteo. Server získaná data ve formátu JSON rozparsuje, vyextrahuje aktuální teplotu a uloží ji do paměti. Vzhledem k tomu, že 8bitové Arduino nepodporuje šifrování HTTPS (nemá dostatek výkonu pro kryptografii TLS/SSL), plní zde dedikovaný server kritickou funkci překladače.

Upozornění a notifikace: Zařízení dokáže upozorňovat na notifikace z chytrého mobilního telefonu. Pro tento účel byl ve Flask aplikaci vytvořen koncový bod POST /api/notification. V ekosystému Apple byla využita aplikace Zkratky (Shortcuts). Pomocí definované automatizace je při obdržení notifikace v telefonu vytvořen a odeslán HTTP POST požadavek na lokální adresu serveru obsahující název aplikace a text zprávy. Server tyto hodnoty zadrží a při následující periodické žádosti od Arduina je zakomponuje do odpovědi k vizualizaci.

1.5 Akustická signalizace a zpracování zvuku

Pro realizaci budíku a notifikací bylo zařízení vybaveno akustickým výstupem (reproduktorem). Výstupní digitální piny mikrokontroléru ATmega2560 jsou z hlediska zatížitelnosti schopny poskytnout proud maximálně 40 mA, což na přímé rozkmitání cívky reproduktoru nestačí.

1.5.1 Pulzně šířková modulace (PWM) a obvod LM386

Pro generování zvuku (tónů různé výšky) se využívá vestavěný čítač a časovač mikrokontroléru. Měněním frekvence, se kterou se digitální výstup přepíná mezi stavy LOW (0 V) a HIGH (5 V), vzniká obdélníkový signál požadovaného kmitočtu. Tento princip vychází z metod pulzně šířkové modulace (PWM).

Tento signál představuje pouze slabé informační napětí. Pro vybuzení 8ohmového reproduktoru je ve výrobku zakomponován modul se zesilovačem zvuku LM386. Jde o nízkonapěťový výkonový operační zesilovač třídy AB. Modul je primárně navržen pro napájení z napětí 5 V (nebo z baterií), má vysoký vstupní odpor a velmi malou spotřebu v klidovém stavu. Napěťové zesílení (Gain) je v základním zapojení interně nastaveno na hodnotu 20 (přidáním kondenzátoru a rezistoru lze zvýšit až na 200). Signál z Arduina je tedy přiveden na vstup modulu a LM386 dodá na svém výstupu adekvátní proud pro dynamický reproduktor.



Obr.5 – LM386 modul pro zesílení signálu

1.6 Napájení a energetická bilance

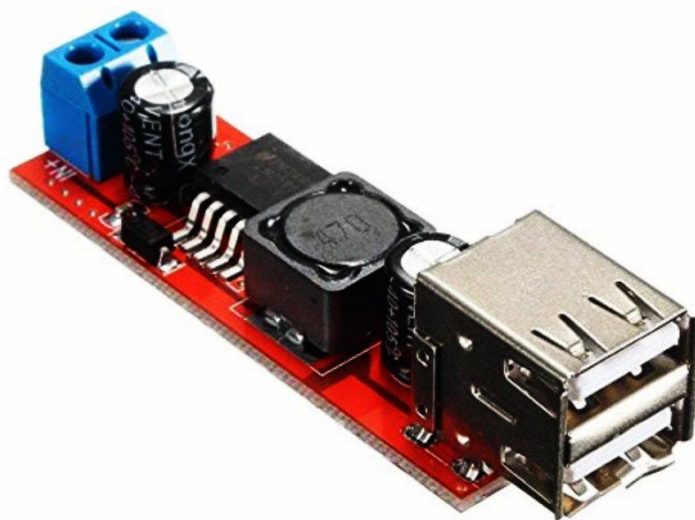
Správné dimenzování napájecí soustavy je pro zařízení běžící v nepřetržitém provozu 24/7 kritickým bodem. Systém jako celek je napájen síťovým adaptérem poskytujícím stejnosměrné napětí 12 V. Z tohoto napětí je ovšem nutné účinně generovat napětí 5 V, které vyžaduje arduino a moduly displejů.

1.6.1 Spínané měniče napětí a obvod LM2596

Ačkoliv má samotná deska Arduino Mega na sobě instalovaný lineární stabilizátor napětí z rodiny 7805, jeho využití by bylo zcela destruktivní. U lineárních stabilizátorů se rozdíl mezi vstupním (12 V) a výstupním (5 V) napětím transformuje výhradně na teplo. Při teoretickém odběru obou displejů a další elektroniky (např. 1 A), by výkonová ztráta (daná vzorcem $P = (U_{in} - U_{out}) \cdot I$) představovala celých 7 W. Běžný lineární regulátor na Arduino, postrádající adekvátní chladič, by po několika minutách zhořel.

Z tohoto důvodu je do projektu zařazen samostatný externí step-down modul založený na monolitickém obvodu LM2596 (v konfiguraci s přímo vyvedeným USB konektorem, produkt prodáváný jako DC-DC měnič s USB) [8].

Jedná se o pulzní (spínaný) měnič pracující na frekvenci 150 kHz. Princip funkce spínaného zdroje (Buck converter) spočívá v rychlém spínání a odpínání vstupního napětí tranzistorem, přičemž energie se ukládá do magnetického pole sériově zapojené tlumivky a vyhlazuje se kondenzátorem. Díky tomu LM2596 dokáže obsloužit výstupní proud až 3 A při účinnosti běžně převyšující 70 až 80 %. Ztrátový výkon (teplo) je tak zredukován na nezbytné minimum a napájení zůstává stabilní i při rozsvícení všech LED na maticích.



Obr.6 - DC/DC step-down měnič s LM2596

1.6.2 Technologie bezdrátového nabíjení (Qi)

Z důvodu zvýšení hodnoty hodin jako doplňku na noční stolek nebo pracovní stůl byla do horní plochy zakomponována cívka pro bezdrátové nabíjení mobilních telefonů. Tato technologie staví na principu Faradayova zákona elektromagnetické indukce. Vysílací cívka zabudovaná ve výrobku generuje měnící se magnetické pole s vysokou frekvencí (dle standardu Qi v pásmu cca 110 až 205 kHz) [9]. Toto pole pak indukuje střídavé napětí v přijímací cívce umístěné na zadní straně mobilního telefonu. Aby byl přenos efektivní a nedocházelo k jeho utlumení, elektronika modulu a vysílací cívka jsou napájeny odděleně, paralelně na stabilních 5 V získaných z měniče LM2596.

1.7 Mechanická konstrukce a 3D tisk

Všechny vnitřní elektronické komponenty vyžadovaly praktické, funkční a estetické opláštění. Zařízení muselo pevně ukotvit oba bloky LED displejů do definovaných úhlů, uchovat velkou desku Arduino Mega společně s reproduktorem a minimalizovat prostor pro průchod drátového propojení. To bylo realizováno technologií 3D tisku.

1.7.1 Návrh v softwaru SolidWorks

Pro samotný mechanický návrh šasi a krytů byl vybrán profesionální parametrický CAD (Computer-Aided Design) software SolidWorks. Proces modelování započal pečlivým změřením rozměrů všech fyzických komponent (posuvným měřítkem) a vytvořením jejich obálkových modelů ve virtuálním prostředí. Poté se pomocí skic, objemových prvků (Extrude), děr a kvót postupně modelovala krabička s přesně definovanými tolerancemi pro "nacvaknutí" nebo zašroubování součástí.

Výraznou inženýrskou výzvou bylo skloubení bezdrátového nabíjení s pevnou konstrukcí plastu. Elektromagnetické pole potřebné pro indukci slábne úměrně k druhé mocnině vzdálenosti. Proto musela být vrchní plocha krytu v SolidWorks odlehčena (vybrána) do přesné síly několika milimetrů, aby nedošlo ke ztrátě dosahu Qi cívky k zadům telefonu. Z modelovacího programu pak byla exportována geometrie ve formátu STL pro generování instrukcí k tisku.

1.7.2 Technologie FDM a materiál PLA

Samotná výroba mechanických součástí z exportovaného modelu proběhla na tiskárně značky Prusa Mini+. Tiskárna operuje na principu technologie FDM (Fused Deposition Modeling) [10]. Plastový polotovar ve formě tenké struny (filament) je zaveden do topného tělesa (hotendu), kde se roztaví. Tavenina je následně pod vysokým tlakem protlačována tryskou o průměru zpravidla 0,4 mm. Tisková hlava se pohybuje po souřadnicích X, Y a nanáší vrstvu po vrstvě roztaveného plastu na podložku, která postupně klesá v ose Z. Pro výrobu šasi byl použit materiál PLA (Kyselina polymléčná - Polylactic acid).

- **PLA bylo vybráno z mnoha důvodů:** Snadnost tisku a tvarová stálost: Materiál se nekrouť tak markantně jako plasty na bázi ropy (ABS), nevyžaduje ke své produkci vyhřívanou komoru tiskárny a nabízí precizní detaily pro lícování součástí šasi.
- **Ekologie:** Jde o termoplastický alifatický polyester, který se získává z obnovitelných zdrojů (kukuřičný nebo bramborový škrob), nikoliv ze surové nafty. Díky tomu je částečně biologicky odbouratelný.

Značnou nevýhodou PLA je jeho poměrně nízká termální rezistence, kdy plast začíná ztrácet strukturální pevnost a měknout (tzv. teplota skelného přechodu) již okolo 55 až 60 °C. Protože ale digitální hodiny představují interiérové elektronické zařízení, které není po delší časové úseky vystavováno přímému slunečnímu svitu (např. v interiéru automobilu) a vzhledem k efektivitě step-down měniče LM2596, byla tato nevýhoda shledána jako nepodstatná..



Obr.7 – Prusa mini+

2 Praktická část – návrh a realizace

Cílem praktické části bylo přetavit výše zmíněné teoretické poznatky do funkčního hardwarového a softwarového řešení. Tato kapitola detailně popisuje architekturu systému, přesné schéma zapojení, postupy při vývoji serverové aplikace a firmwaru pro mikrokontrolér a nakonec proces mechanického modelování šasi.

2.1 Blokové schéma a celková architektura systému

Celková architektura projektu je navržena tak, aby docházelo k maximální optimalizaci toku dat a rozložení zátěže.

Z hlediska napájení tvoří primární zdroj externí síťový adaptér poskytující stabilizované stejnosměrné napětí 12 V. Toto napětí je vedeno přímo na vstup step-down měniče LM2596. Měnič transformuje napětí na bezpečnou úroveň 5 V s vysokou účinností. Z tohoto uzlu se napájení rozvětňuje do tří nezávislých větví:

1. Napájení vývojové desky Arduino Mega 2560 (přes USB).
2. Napájení dvou LED maticových displejů (zabraňuje se tím přetížení stabilizátoru na samotném Arduino).
3. Napájení modulu pro bezdrátové nabíjení standardu Qi.

Z hlediska datových toků je mozkiem operací dedikovaný server. Ten komunikuje do vnější sítě internetu přes šifrované HTTPS protokoly (stahování počasí z Open-Meteo) a zároveň naslouchá na portu 5000. Mikrokontrolér Arduino je k serveru připojen přes Ethernetový modul a s frekvencí 10 sekund se dotazuje na aktuální sestavu dat k zobrazení.

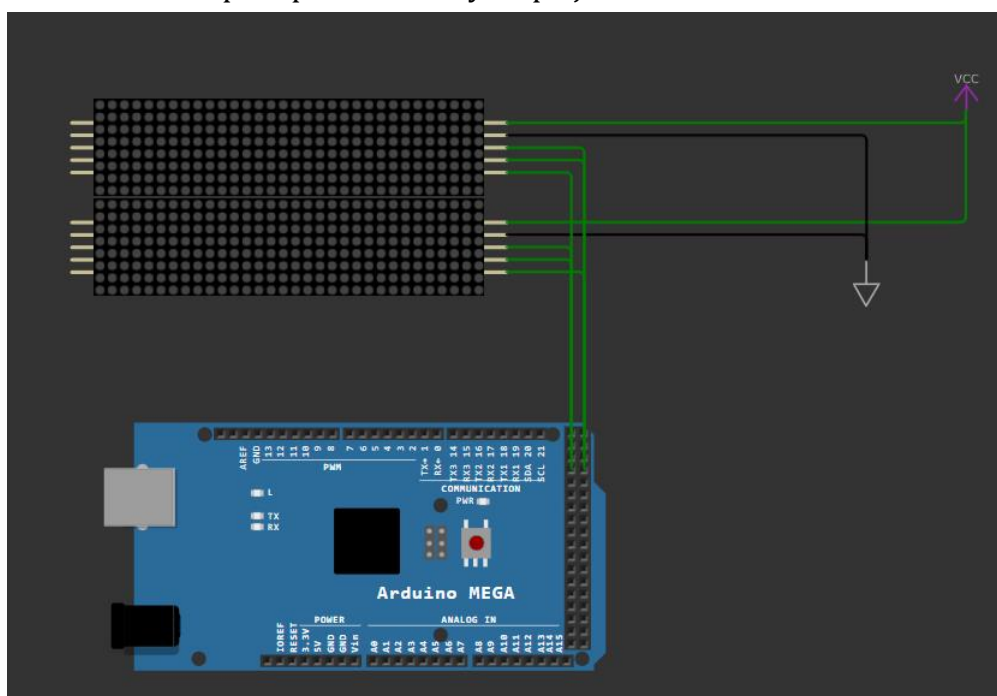
2.2 Hardwarová realizace a zapojení komponent

Fyzické propojení komponent nevyžadovalo výrobu vlastního plošného spoje (PCB), neboť většina prvků existuje ve formě integrovaných modulů připravených pro propojovací vodiče.

Stěžejní bylo správné rozložení digitálních pinů na desce Arduino Mega, a to zejména pro zamezení kolizí na SPI sběrnici:

- **Ethernet Shield W5100:** Tento modul je osazen přímo jako rozšiřující karta navrch desky Arduino Mega. Komunikuje přes hardwarové SPI piny (MISO 50, MOSI 51, SCK 52). Jako Chip Select (CS) je pevně určen pin 10. Pro zajištění bezproblémového bootu je v kódu navíc vynuceno trvalé vypnutí interní čtečky SD karet (která je součástí shieldu) přivedením logické úrovně HIGH na pin 53.
- **Modul displeje 1 (horní - Čas):** Komunikace probíhá softwarově emulovaným SPI pomocí knihovny LedControl. Datový pin (DIN) připojen na pin 22, hodinový signál (CLK) na pin 23, volba čipu (CS) na pin 24.
- **Modul displeje 2 (dolní - Teplota/Datum):** Datový pin (DIN) připojen na pin 25, hodinový signál (CLK) na pin 26, volba čipu (CS) na pin 27.
- **Zvukový výstup:** Aktivní prvek pro generování PWM signálu je připojen na pin 8, který vede na vstup zesilovače LM386 a následně do 8ohmového reproduktoru.

Rozdělení displejů do dvou nezávislých větví (namísto kaskádového propojení 8 modulů do jednoho řetězce) se ukázalo jako klíčové hardwarové rozhodnutí pro zajištění plynulého posouvání notifikací na horním displeji, aniž by se překreslování zpomalovalo nutností odesílat data i pro spodní statický displej.



Obr.8 – zapojení LED displejů a Arduino Mega 2560

2.3 Vývoj serverové aplikace (Backend)

Serverová aplikace tvoří mozek celého IoT ekosystému. Je napsána v programovacím jazyce Python 3. Pro její běh je využit framework Flask, který umožňuje rychlé a přehledné vystavení REST API.

2.3.1 Získávání meteorologických dat

Získávání teploty je realizováno asynchronně vůči hlavním webovým procesům. Ve skriptu app.py je definováno vlastní vlákno (knihovna threading), které v nekonečné smyčce volá funkci `fetch_temperature()` a následně se na 900 sekund (15 minut) uspí. Funkce odesílá HTTP GET požadavek s GPS souřadnicemi Liberce na bezplatné API služby Open-Meteo. Příchozí odpověď typu JSON je rozparsována a aktuální teplota je zapsána do globální stavové proměnné `current_temperature`, ze které následně čerpají další funkce. Tento návrhový vzor zabraňuje zablokování serveru v případě výpadku konektivity k meteorologické službě.

```
# ----- Konfigurace -----  
# Liberec: 50.7671 N, 15.0562 E  
OPEN_METEO_URL = (  
    "https://api.open-meteo.com/v1/forecast"  
    "?latitude=50.7671&longitude=15.0562"  
    "&current_weather=true"  
    "&timezone=Europe%2FPrague"  
)  
TEMP_FETCH_INTERVAL = 900 # 15 minut  
PRAGUE_TZ = ZoneInfo("Europe/Prague")
```

2.3.2 Správa notifikací a aplikační rozhraní

Pro interakci s chytrým telefonem byl vytvořen endpoint obsluhovaný funkcí pod dekorátorem `@app.route("/api/notification", methods=["POST"])`. Funkce je navržena vysoce robustně, aby dokázala přijmout data jak ve formátu JSON, tak jako form-data či pouhý text (plain-text).

Aplikace Zkratky (Shortcuts) v operačním systému iOS je nastavena tak, že při zachycení libovolného příchozího upozornění vytvoří slovník (Dictionary), kde klíčem je název aplikace (např. "Instagram") a hodnotou je obsah zprávy. Tento JSON odesílá POST metodou do serveru, který jej rozloží do proměnných `notification_title` a `notification_body`.

Při zavolání hlavního endpointu Arduinem na cestě `/api/arduino` vezme systém všechny hodnoty (přesný čas převedený do časové zóny Prahy, stav budíku, počasí a přijatou zprávu) a složí je do jednoduchého textového řetězce ohraničeného zobáčkovými závorkami a oddělovači typu "pipe" (`|`).

`<TIME:15:30|DATE:05.10.|TEMP:22|ALARM:0|MSG:Nová zpráva>.`

2.4 Vývoj firmwaru pro mikrokontrolér

Program pro Arduino Mega (napsaný v jazyce C++) je navržen s ohledem na maximální spolehlivost a ochranu omezené paměti mikrokontroléru.

2.4.1 Inicializace sítě a HTTP komunikace

V bloku `setup()` dochází k inicializaci modulu W5100. Pomocí volání `Ethernet.begin(mac)` se zařízení pokusí získat IP adresu dynamicky pomocí DHCP. V případě neúspěchu přejde zařízení do nekonečné smyčky, aby předešlo neočekávanému chování.

Hlavní smyčka `loop()` je koncipována výhradně jako asynchronní (neblokující) pomocí funkce `millis()`. Tím je zaručeno, že dotazování serveru (nastavené na 10 sekund) nezastaví scrollování textu na maticích nebo pípání budíku, jako by tomu bylo při použití blokující funkce `delay()`.

```
void setup() {  
  
    ...  
  
    if (Ethernet.begin(mac) == 0) {  
        Serial.println(F("[NET] DHCP selhalo!"));  
        if (Ethernet.hardwareStatus() == EthernetNoHardware) {  
            Serial.println(F("[NET] Ethernet shield nenalezen!"));  
        }  
        if (Ethernet.linkStatus() == LinkOFF) {  
            Serial.println(F("[NET] Kabel není zapojen!"));  
        }  
        displayBootText("ERR");  
        while (true) { delay(1000); }  
    }  
}
```

Ukázka inicializace ethernet modulu

```
void loop() {  
    unsigned long now = millis();
```

Ukázka hlavní smyčky loop()

2.4.2 Parsování datových struktur bez použití objektu String

Velmi častou chybou začínajících vývojářů na platformě Arduino je masivní využití vestavěného objektu String. Tento objekt při každé spojovací operaci alokuje a uvolňuje nové místo v paměti SRAM (heap), což vede po krátkém čase k fragmentaci paměti a zamrznutí procesoru.

V této práci se objekt String záměrně nikde nevyskytuje. Příchozí odpověď od serveru se ukládá postupně znak po znaku (char) do pole pevné délky (char responseBuf[256]). Následné vyhledávání hodnot v tomto poli je realizováno pomocí ukazatelů (pointerů) a standardních C funkcí strstr a memcpy. Funkce extractValue vyhledá klíč (např. "TEMP:"), posune ukazatel na začátek hodnoty a kopíruje znaky až po narazení na ohraničující znak ("|" nebo ">"). Tímto způsobem spotřeba paměti nikdy nekolísá a zařízení dokáže běžet měsíce bez nutnosti restartu.

2.4.3 Správa paměti a vykreslování na LED matice

Pro zobrazení libovolného textu bylo nutné vytvořit vlastní znakovou sadu (font 5x7 bodů). Běžné uložení takového pole do proměnných by spotřebovalo stovky bajtů operační paměti. Využilo se tedy direktivy PROGMEM, která kompilátoru sděluje, že obrovské binární pole znaků (v kódu deklarováno jako const uint8_t PROGMEM font5x7[]) má zůstat uloženo pouze ve Flash paměti (určené primárně pro kód programu) a bude se odtud číst jen v okamžiku potřeby pomocí makra pgm_read_byte.

K samotnému vykreslování existují softwarové buffery fb1 a fb2 (každý o velikosti 32 bajtů reprezentujících 32 sloupců). Funkce drawString vloží pixely konkrétních písmen do tohoto bufferu a funkce sendFB následně tyto bity transponuje (otočí o 90 stupňů) a odešle pomocí knihovny LedControl do obvodů MAX7219.

```
const uint8_t PROGMEM font5x7[] = {  
    // 32: (space)  
    0x00,0x00,0x00,0x00,0x00,  
    // 33: !  
    0x00,0x00,0x5F,0x00,0x00,  
    // 34: "  
    0x00,0x07,0x00,0x07,0x00,  
    // 35: #  
    0x14,0x7F,0x14,0x7F,0x14,  
    // 36: $  
    0x24,0x2A,0x7F,0x2A,0x12,  
    // 37: %  
    0x23,0x13,0x08,0x64,0x62,  
    // 38: &  
    0x36,0x49,0x55,0x22,0x50,  
    // 39: '  
    0x00,0x05,0x03,0x00,0x00,  
    // 40: (  
    0x00,0x1C,0x22,0x41,0x00,  
    // 41: )  
    0x00,0x41,0x22,0x1C,0x00,  
    // 42: *  
    0x08,0x2A,0x1C,0x2A,0x08,  
    // 43: +  
    0x08,0x08,0x3E,0x08,0x08,  
    // 44: ,  
    0x00,0x50,0x30,0x00,0x00,  
    // 45: -  
    0x08,0x08,0x08,0x08,0x08,
```

Ukázka zapsání fontu v kódu

2.5 Mechanický návrh a výroba šasi

Návrh krabičky byl proveden v systému SolidWorks. Celé zařízení je rozděleno do dvou nosných celků: spodní základna s průzory pro displeje a vrchní odnímatelný kryt.

Základna obsahuje čtyři distanční sloupky pro připevnění desky Arduino Mega pomocí M3 šroubů. Zároveň je v ní integrován slot pro vyvedení napájecího kabelu a ethernetového kabelu od Ethernet shieldu.

Základna má v přední stěně dva obdélníkové výřezy přesně lícující s deskami plošných spojů LED matic. Důležitým konstrukčním prvkem jsou vodící drážky pro zasunutí obou panelů, aby nebylo nutné používat lepidlo, což by znemožnilo případný servis.

Nejsložitějším dílem byl vrchní kryt. Pro splnění požadavku bezdrátového nabíjení (standard Qi) byla spodní strana krytu vyfrézována (pomocí operace Vykrytí/Kapsa v SolidWorks) a zanechala tak stěnu o tloušťce pouhých 1,6 mm (4 vrstvy při tisku s tloušťkou vrstvy 0,4 mm). Tato tloušťka zaručuje, že magnetické pole bez problému prostoupí do přijímací cívky smartphonu položeného na hodinách. K tisku byl využit materiál PLA z důvodu jeho rozměrové přesnosti a nízké míry smršťování.

3 Testování a měření

Tato fáze projektu se zaměřila na praktické ověření funkčnosti navrženého zařízení a zejména na jeho energetickou bilanci. V případě IoT zařízení je znalost spotřeby energie klíčová pro dimenzování napájecích zdrojů a chladičů.

3.1 Oživení systému a kalibrace

Během ožívání systému byl kód nejprve nahrán na nepájivém kontaktním poli (Breadboard), aby byla ověřena stabilita SPI sběrnice a správná konfigurace Ethernet shieldu. Po připojení do lokální sítě Arduino úspěšně získalo IP adresu z domácího routeru pomocí DHCP. PING odezva od zařízení se stabilně pohybovala do 2 ms.

Následně byl otestován Python server. Úspěšně byla ověřena integrace služby Open-Meteo, kdy logy serveru potvrdily stažení JSON dat a jejich přepis do proměnné každých 15 minut. Během testování notifikací z iPhoneu bylo zjištěno, že Zkratky v iOS někdy formátují data nestandardně; proto byla ve skriptu přidána rozsáhlá sekce využívající try-except bloky, která dokáže rozluštit jak formát JSON, tak běžná data zaslaná jako form-data nebo pouhý text.



Obr. 9 – testovací zobrazení času a teploty na displejích



Obr.10 – testovací zobrazení času a datumu na displejích

3.2 Měření energetické spotřeby

Energetické požadavky zařízení byly měřeny multimetrem zapojeným v sérii na větvi mezi USB převodníkem a hlavní sběrnici. Měření bylo rozděleno do několika scénářů simulujících reálný provoz.

Spotřeba samotného řídicího modulu (Arduino + Ethernet):

Při analýze základního odběru mikrokontroléru byly získány následující hodnoty:

- **Fáze startu a inicializace sítě (Boot):** 140 mA
- **Normální běh s aktivním Ethernet spojením:** 120 mA

Spotřeba zobrazovacích jednotek (MAX7219 matice):

LED matice patří mezi energeticky nejnáročnější komponenty systému. Měření ukázalo výraznou závislost odebíraného proudu na počtu aktivních bodů a softwarově nastaveném jasu:

- **Displeje vypnuté (reps. režim spánku po 10 min. bez aktivity):** odběr klesl na ~0 mA pro samotné displeje (celý systém vykazoval odběr pouhých 120 mA).
- **Zobrazený běžný text (Čas a teplota na střední jas):** odběr sestavy (2 displeje) činil průměrně 400 mA.
- **Plné rozsvícení všech 512 LED diod na maximální jas:** odběr vysoce přesahoval předepsaných 500 mA, až na průměrných 750 mA.

Z těchto měření jednoznačně vyplývá opodstatněnost použití výkonného spínaného měniče LM2596. Pokud by zařízení v plném zatížení (120 mA mozek + 750 mA displeje = cca 870 mA) běželo z lineárního stabilizátoru napájeného 12 V, ztrátový výkon by převyšoval 6 W. Tento výkon by stabilizátor v arduinu nedokázal. Pulzní měnič LM2596 s účinností kolem 80 % však tuto transformaci zvládá bez zjevného nárůstu teploty na svých chladičích plochách, a navíc mu zbývá obrovská kapacita (až 2 Ampéry) pro obsluhu integrované bezdrátové nabíječky Qi, která se aktivuje po položení telefonu na horní kryt.

3.3 Testování softwarové stability

Klíčovou podmínkou pro funkční digitální hodiny je stabilita bez výpadků. Zařízení bylo podrobeno vícedennímu testu, kdy běželo 7 dní bez resetu. Arduino se každých 10 sekund spojovalo s Python serverem. V rámci programového zajištění Arduino úspěšně ignorovalo timeouty (pokud server nestihl odpovědět do 3 sekund, Arduino spojení ukončilo bez zablokování chodu plynulého scrollování notifikací).

Rovněž byla otestována implementace režimu spánku (Sleep Mode), kdy systém zjistí zda na server v posledních 10 minutách přišla nová notifikace Po 10 minutách bez nové notifikace Arduino posílá obvodům MAX7219 povel k vypnutí (přechod do stavu Shutdown s nulovým odběrem proudu) a čeká. Jakmile získá znovu platná data, displeje ihned aktivuje a pokračuje v normálním provozu.

Závěr

Cílem této maturitní práce byl vývoj a konstrukce funkčního koncového zařízení internetu věcí ve formě multifunkčních digitálních hodin, které by přesahovaly rámec běžně dostupné spotřební elektroniky svými pokročilými síťovými možnostmi.

Tohoto cíle bylo v plném rozsahu dosaženo. Byl navržen, vytištěn a osazen prototyp hodin, který vizuálně sdružuje informace o přesném času, teplotě vzduchu a příchozích notifikacích uživatele. Provedená práce úspěšně zkombinovala široké spektrum technologických oborů: od nízkoúrovňového programování C++ u mikrokontroléru Arduino Mega přes návrh asynchronního webového API ve frameworku Python Flask, modelování ve 3D CAD programu SolidWorks až po studium napájecích účinností pulzních měničů LM2596 a principů elektromagnetické indukce v bezdrátových nabíječkách standardu Qi.

Největší inovací a odklonem od původního zadání bylo ustoupení od fyzických senzorů teploty a jejich nahrazení sofistikovanější cloudovou architekturou klient-server, která umožnila začlenit zařízení hlouběji do chytré domácnosti (Smart Home) bez nutnosti fyzických externích jednotek a řešení jejich napájení či rádiového propojení. Serverová integrace dat pomocí Open-Meteo API a iOS Apple Shortcuts se prokázala jako vysoce spolehlivá.

Energetické testování prototypu potvrdilo robustnost zvolené napájecí větve a stabilita softwarového zpracování zaručila plynulý chod bez paměťových úniků (Memory Leaks) díky důmyslnému parsování přes ukazatele a využití PROGMEM direktivy pro paměťově náročné fonty.

Výsledkem práce je elegantní a spolehlivý kus hardwaru, který může do budoucna sloužit jako otevřená platforma pro další implementaci chytrých domácích automatizací, připojení na asistenty typu Home Assistant nebo rozšíření funkčnosti pomocí audio signálů širšího spektra.

Seznam zkratk a odborných výrazů

- **IoT (Internet of Things):** Internet věcí, síť propojených fyzických zařízení komunikujících navzájem.
- **MCU (Microcontroller Unit):** Jednočipový počítač, mikrokontrolér, který tvoří "mozek" zařízení.
- **SRAM (Static Random Access Memory):** Rychlá operační paměť s náhodným přístupem, ztrácí data po odpojení.
- **SPI (Serial Peripheral Interface):** Sériové periferní rozhraní, synchronní komunikační sběrnice typu Master-Slave.
- **TCP/IP (Transmission Control Protocol / Internet Protocol):** Sada protokolů tvořící páteř síťové komunikace.
- **DHCP (Dynamic Host Configuration Protocol):** Protokol pro automatické přidělování IP adres počítačům či IoT zařízením v síti.
- **API (Application Programming Interface):** Rozhraní pro programování aplikací, přesné definování pravidel komunikace mezi softwary.
- **PWM (Pulse Width Modulation):** Pulzně šířková modulace, změna střídy (času sepnutí) obdélníkového signálu pro např. generování tónů, regulaci svitu či otáček.
- **CAD (Computer-Aided Design):** Počítačem podporované projektování pro tvorbu 3D modelů (zde zastoupené programem SolidWorks).
- **FDM (Fused Deposition Modeling):** Nejrozšířenější technologie 3D tisku postupným nanášením a vrstvením roztaveného plastového vlákna.
- **PLA (Polylactic Acid):** Kyselina polymléčná, ekologický bioplast nejpoužívanější pro komerční a domácí 3D tisk.
- **JSON (JavaScript Object Notation):** Oblíbený a lehký datový formát nezávislý na jazyce, používaný pro výměnu strukturovaných textových dat.
- **Wiring:** Open-source programovací framework (vývojový rámec), na kterém je softwarově založena platforma Arduino. Zjednodušuje nízkoúrovňovou komunikaci s hardwarem do srozumitelných příkazů v jazyce C/C++.

Seznam obrázků

Obr.1 – blokové schéma architektury mikrokontrolerů.....	3
Obr.2 – Arduino Mega 2560	4
Obr.3 – LED Matrix matice 8x8x4 s MAX7219	5
Obr.4 – Ethernet shield pro Arduino.....	8
Obr.5 – LM386 modul pro zesílení signálu.....	11
Obr.6 – DC/DC step-down měnič s LM2596	12
Obr.7 – Prusa mini+	14
Obr.8 – zapojení LED displejů a Arduino Mega 2560	16
Obr.9 – testovací zobrazení času a teploty na displejích.....	23
Obr.10 – testovací zobrazení času a datumu na displejích.....	23

Použité zdroje

- [1] WIKIPEDIE. *Microcontroller (mikrokontrolér)*. Online. 2026. Dostupné z: <https://en.wikipedia.org/wiki/Microcontroller>. [cit. 2026-01-26].
- [2] BRITANNICA. *Internet of Things (IoT)*. Online. 2026. Dostupné z: <https://www.britannica.com/science/Internet-of-Things>. [cit. 2026-01-26].
- [3] ARDUINO. *Arduino documentation — IoT and networking examples*. Online. 2026. Dostupné z: <https://docs.arduino.cc/>. [cit. 2026-01-26].
- [4] WIKIPEDIE. *Liquid-crystal display (LCD) / LED Matrix*. Online. 2026. Dostupné z: https://en.wikipedia.org/wiki/LED_display. [cit. 2026-01-26].
- [5] MAXIM INTEGRATED. *MAX7219 Datasheet*. PDF. 2003. Dostupné z: <https://www.analog.com/media/en/technical-documentation/data-sheets/max7219-max7221.pdf>. [cit. 2026-03-15].
- [6] WIZNET. *W5100 Datasheet*. PDF. 2008. Dostupné z: <https://www.google.com/search?q=https://docs.wiznet.io/Product/iEthernet/W5100/datasheet>. [cit. 2026-03-15].
- [7] PALLETS PROJECTS. *Flask Documentation (3.0.x)*. Online. 2026. Dostupné z: <https://flask.palletsprojects.com/>. [cit. 2026-03-15].
- [8] TEXAS INSTRUMENTS. *LM2596 SIMPLE SWITCHER Power Converter High Efficiency 3A Step-Down Voltage Regulator*. PDF. 2021. Dostupné z: <https://www.ti.com/lit/ds/symlink/lm2596.pdf>. [cit. 2026-03-15].
- [9] WIRELESS POWER CONSORTIUM. *Qi Specification*. Online. 2026. Dostupné z: <https://www.wirelesspowerconsortium.com/qi/>. [cit. 2026-03-15].
- [10] PRUSA RESEARCH. *Základy 3D tisku s Josefem Průšou*. Praha: Prusa Research, 2023. ISBN 978-80-908077-0-0.